

Kapitel 3

Beispielprogramme

Nachdem wir die Standard-Datentypen von C ausführlich besprochen haben, sollen jetzt zunächst mehr informell einige Möglichkeiten der Ein-/Ausgabe und zwei Anweisungen zur Steuerung des Programmablaufs besprochen werden, damit erste einigermaßen sinnvolle Programme entstehen können. Wir werden auf das meiste später auch noch ausführlicher zurückkommen.

3.1 Formatierte Ein-/Ausgabe

Wir haben bereits gesehen: C selbst kennt zwar keine Anweisungen zur Ein-/Ausgabe, jedoch stellt die Standardbibliothek entsprechende Funktionen zur Verfügung, die dem Compiler durch die Präprozessor-Direktive

```
#include <stdio.h>
```

am Anfang einer Quelldatei bekanntgemacht werden können bzw. müssen, wenn man Daten lesen oder schreiben will.

Die Ausgabefunktion `printf` haben wir in unserem ersten Beispiel bereits kennengelernt, wenn auch nur in extrem elementarer Form. Sie schreibt immer auf das *Standardausgabegerät*, in der Regel also auf den Bildschirm. Ihr Pendant für die Eingabe ist die Funktion `scanf`, die stets vom *Standardeingabegerät* liest, in der Regel also von der Tastatur.

Streng genommen muss man `scanf` und `printf` als Funktionen zur *formatierten Ein-/Ausgabe* bezeichnen. Darunter versteht man, dass im Zuge der Ein-/Ausgabe numerischer Werte eine Umwandlung zwischen *interner* (binärer) und *externer* Darstellung erfolgt. Wir sehen uns das an einem Beispiel an:

```
#include <stdio.h>

/*= Hauptprogramm =====*/
int main(void) {
    int i;

    scanf("%d", &i);                /* Wert lesen      */
    printf("%d", i);                /* Wert ausgeben   */
    return 0;
}
```

Quelltext 3.1: einfache formatierte Eingabe/Ausgabe

Zu den Formalien zunächst nur soviel: Die *Formatstrings* `"%d"` bei beiden Funktionen zeigen an, dass ein Wert mit dem Typ `signed int` zu lesen bzw. zu schreiben ist. Der Operator `&` bei `scanf` zeigt an, dass ein Wert in der Variablen zu speichern ist. Auf beides werden wir gleich noch näher eingehen.

Nun aber zunächst zum Ablauf von Übertragung und Umwandlung:

- `scanf` fordert Eingabe von der Tastatur an. Die Tastatur liefert (Ordnungszahlen von) Zeichen.
- Der gelesene Wert soll in einer `int`-Variablen gespeichert werden. Deshalb müssen die Zeichen interpretiert und in die entsprechende binäre Darstellung umgewandelt werden. Wie viele Bits erzeugt werden müssen, hängt nur von der Anzahl der Bytes ab, die zur Speicherung von `int`-Werten für die Rechnerarchitektur festgelegt ist und hängt nicht von der Größe des Wertes ab.
- `printf` soll einen `int`-Wert schreiben, der in Binärdarstellung vorliegt. Da der Bildschirm mit der Binärdarstellung nichts anzufangen weiß, muss sie zunächst in Zeichen umgewandelt werden.
- Die Zeichen werden auf den Bildschirm übertragen.

Tatsächlich ist der Ablauf sehr viel komplizierter. Da die zusätzlichen Arbeiten aber stets durch elementare Routinen automatisch erledigt werden, braucht uns das nicht zu interessieren.

3.2 Formatierte Ausgabe

Schauen wir uns jetzt zunächst die Ausgabefunktion `printf` näher an, weil sie etwas einfacher als die Eingabefunktion ist. Wir haben sie im ersten Abschnitt dieses Kapitels bereits verwendet.

```
printf("%d", i);
```

Zur Funktionsweise müssen aber zunächst einige Dinge geklärt werden: Funktionen erhalten grundsätzlich extrem wenig Information über die Argumente aus dem Aufruf, nämlich entweder den Wert oder die Speicheradresse. Der Typ der Argumente kann beim Aufruf nicht mitgeteilt werden.

Das ist in der Regel kein Problem, weil die Funktionen die Typen ihrer Argumente meist selbst wissen: Eine Funktion zur Berechnung einer Winkelfunktion kann zum Beispiel sinnvollerweise nur eine Gleitkommazahl als Argument erhalten; sieht man für die drei verschiedenen Gleitkommatypen unterschiedliche Funktionen vor, so weiß jede der Varianten genau, welchen Typ ihr Argument hat.

Bei den Funktionen für die Ein-/Ausgabe sieht das anders aus: Es wäre extrem lästig, wenn man für jeden Typ eine andere Funktion aufrufen müsste. Deshalb geht man anders vor. Nur das erste Argument ist festgelegt und damit den Funktionen bekannt; es muss nämlich ein String sein. Allerdings darf es nicht ein beliebiger String sein, sondern es muss ein Formatstring sein. Damit ist gemeint, dass der String die weiteren Argumente beschreibt, und zwar sowohl die Anzahl als auch die Typen der Argumente.

Jede Beschreibung eines Arguments im Formatstring wird durch ein Prozentzeichen (%) eingeleitet. Ihm muss ein Kennbuchstabe (oder auch eine Kennbuchstaben-Kombination) für den Typ des Wertes folgen; dazu können weitere Angaben kommen, die uns zum jetzigen Zeitpunkt aber noch nicht zu interessieren brauchen. Jede dieser Angaben wird als *Formatbeschreiber* bezeichnet.

	“Standard”-Typ	Darstellung
i	signed int	ganzzahlig dezimal
d		ganzzahlig dezimal
u	unsigned int	ganzzahlig dezimal
o		ganzzahlig oktal
x		ganzzahlig hexadezimal
X		ganzzahlig hexadezimal
f	double	exponentenfrei, mit oder ohne Dezimalpunkt
e		halblogarithmisch
E	double	exponentenfrei oder halblogarithmisch
G		
c	char	einzelnes Zeichen
s	char *	Zeichenfolge
p	void *	Adresse

Tabelle 3.1: Kennbuchstaben für Ausgabeformate

Fassen wir zusammen: An der Anzahl der Prozentzeichen im Formatstring erkennt **printf** die Anzahl der noch folgenden Argumente. Wie im ersten Programmbeispiel auf Seite 4, kann die Anzahl auch Null sein. Anhand der Kennbuchstaben, die den Prozentzeichen folgen, erkennt **printf**, welche Typen die Argumente haben. Dabei erfolgt die Zuordnung zwischen den Formatbeschreibern und den Ausgabewerten linear. Dass die Formatbeschreiber in Anzahl und Typen den Ausgabewerten entsprechen, ist *ausschließlich* in die Verantwortung des Programmierers gestellt. Verstöße können zu „kuriosen“ Ausgaben führen – oder auch zum „Absturz“ des Programms.

Zurück zu den Formatbeschreibern. So wie es eben beschrieben wurde, wäre für jeden Typ ein anderer Kennbuchstabe oder eine andere Kombination von Kennbuchstaben erforderlich. Das ist aber nicht der Fall, weil bei der Übergabe **float**-Werte grundsätzlich in **double** und **short**-Werte grundsätzlich in **int** umgewandelt werden. Auch **char**-Werte werden grundsätzlich in **int** umgewandelt; der Kennbuchstabe für den Typ **char** bewirkt die Rückumwandlung von **int** nach **char** durch **printf**.

Die Kennbuchstaben sind in Abbildung 3.1 wiedergegeben. Für Argumente mit dem Typ **long int** muss dem jeweiligen **int**-Kennbuchstaben der Zusatz **l**, für Argumente mit dem Typ **long double** dem jeweiligen **double**-Kennbuchstaben der Zusatz **L** vorangestellt werden.

Die Anzahl der Zeichen, die zur Darstellung eines Wertes verwendet werden, richtet sich nach Standard-Vorgaben, wenn man nur die genannten Kennbuchstaben verwendet, was für uns zunächst ausreicht. Wesentlicher ist: Zeichen im Formatstring, die nicht zu einem Formatbeschreiber gehören, werden unverändert in die Ausgabe übernommen. Genau dieses haben wir in unserem ersten Beispiel genutzt.

Schauen wir uns einige Beispiel an. Vereinbarung seien die Variablen

```
int i;
long l;
float f;
double d;
long double ld;
```

Dann können wir programmieren

```
printf ("Ganze Zahlen, dezimal: %d, %ld", i, l);
printf ("Ganze Zahlen, oktal: %o, %lo", i, l);
printf ("Gleitkommazahlen: %e, %E", f, d);
printf ("Gleitkommazahlen: %f, %Lf", f, ld);
```

In allen vier Beispielen erscheint zunächst der Text, dann der Wert der ersten Variablen, dann ein Komma und schließlich der Wert der zweiten Variablen. Wie die Zahlen selbst dargestellt werden? – Schauen Sie es sich am Rechner einmal an!

3.3 Formatierte Eingabe

Sehen wir uns jetzt die Eingabe durch `scanf` an. Der wesentlichste Unterschied zu `printf`: Die Funktion `printf` benötigt nur die *Werte* ihrer Argumente; ob sie Konstanten oder Variablen sind und wo sie ggf. im Speicher stehen, ist gleichgültig. Da der Name einer Variablen in C in den meisten Fällen als Bezeichnung für ihren Wert interpretiert wird, reicht es aus, die Namen der Variablen in den Aufruf von `printf` einzusetzen. Bei `scanf` ist das anders. Diese Funktion muss wissen, *wohin* sie den gelesenen Wert schreiben soll, d.h. sie benötigt nicht die Werte ihrer Argumente, sondern deren *Adressen*. Der *Adressoperator* `&`, der bereits erwähnt und verwendet wurde, sorgt gerade hierfür.

Den Adressoperator bei den Argumenten von `scanf` zu vergessen, ist einer der „beliebtesten“ Fehler – nicht nur bei Anfängern. Wir wollen uns überlegen, was dieser Fehler bewirkt, etwa wenn wir

```
int i = 0;
scanf ("%d", i);
```

schreiben. Da der Operator `&` fehlt, wird der *Wert* von `i` anstelle der *Adresse* von `i` übergeben, hier also 0. Die Funktion `scanf` wird durch den Formatstring informiert, dass ein `int`-Wert zu lesen ist, und unterstellt deshalb, dass das nächste Argument die *Adresse* einer `int`-Variablen ist. Der Versuch, 0 als Adresse zu verwenden, kann in der Regel nur ins Chaos führen. Wenn man Glück hat, überschreibt man nur Daten, die man ohnehin nicht mehr braucht. Nur – Glück hat man in der Regel nicht. Das Programm wird in der Regel mit einem *segmentation fault*, *memory fault* oder *bus error* vom Betriebssystem „gekillt“, hoffentlich ohne Auswirkungen auf andere, gleichzeitig laufende Prozesse.

Im übrigen gilt wie für `printf`: Durch den Formatstring, der als erster Parameter übergeben wird, muss der Funktion mitgeteilt werden, wie viele Argumente folgen und welche Typen diese besitzen; die Verantwortung für die Übereinstimmung von Formatstring und Argumentliste obliegt dem Programmierer. Unterschiede bestehen dabei allerdings:

- Eine eventuelle Typumwandlung wie bei `printf` ist keinesfalls möglich; ist etwa ein Zeichen zu lesen, so umfasst der Speicherplatz der entsprechenden Variablen (in der Regel) ja nur ein Byte, so dass `scanf` auch nur ein Byte an die übergebene Adresse in den Speicher schreiben darf.
- Ob die Eingabezeichen für ganze Zahlen als dezimale, oktale oder hexadezimale Darstellung des Wertes zu interpretieren sind, ist durchaus wesentlich. Offensichtlich gilt ja $11_d \neq 11_o \neq 11_h$. Ebenso ist es wesentlich, ob ein ganzzahliger Eingabewert ein Vorzeichen tragen darf oder nicht. Anders sieht es bei den Gleitkommazahlen aus: Welche der drei möglichen Darstellungen bei der Eingabe verwendet wird, ist für die interne Darstellung der Werte bedeutungslos.
- Zeichen im Formatstring, die nicht zu Formatbeschreibern gehören, müssen bei der Eingabe eingetippt werden. An die rufende Funktion werden sie nicht weitergegeben.

	erwartete Zeichenfolge	“Standard”-Typ
i	ganzzahlig, mit oder ohne Vorzeichen	signed int *
d	ganzzahlig dezimal, mit oder ohne Vorzeichen	
u	ganzzahlig dezimal	unsigned int *
o	ganzzahlig oktal	
x	ganzzahlig hexadezimal	
X	ganzzahlig hexadezimal	
f	Gleitkommawert, mit	float *
e	oder ohne Vorzeichen,	
E	mit oder ohne Dezimal-	
g	punkt, mit oder ohne	
G	Exponententeil	
c	einzelnes Zeichen oder Zeichenfolge	char *
s	Zeichenfolge	char *

Tabelle 3.2: Kennbuchstaben für Eingabeformate

Zweckmäßig ist es deshalb, die Formatbeschreiber durch ein Leerzeichen voneinander zu trennen, da man die Eingabewerte ohnehin auch durch ein Leerzeichen (Zeilenende, o.ä.) trennen muss, andere Zeichen in Eingabeformaten aber nicht zu verwenden.

Die Kennbuchstaben für die „Grundtypen“ sind in Abbildung 3.2 wiedergegeben. Andere Typen müssen durch einen zweiten, vorangestellten Kennbuchstaben gekennzeichnet werden:

- h** **short** bei ganzzahligen Werten
- l** **long** bei ganzzahligen Werten
 double bei Gleitkommawerten
- L** **long double** bei Gleitkommawerten

In den nächsten Abschnitten werden wir in erster Linie Beispiele für die Eingabe betrachten, weil sie letztlich interessanter als die Ausgabe ist. Ausgabe erweist sich, wenn man „schöne“ Ausgaben erzeugen möchte, als reines „Abzählspiel“.

3.4 Schleifen

Als erstes Beispiel wollen wir diese Aufgabe lösen: Ein Programm soll zehn (**int**-)Zahlen lesen und wieder schreiben.

Lösen kann man diese Aufgabe mit den bereits bekannten Sprachelementen auf zwei Weisen:

- Man kann eine Variable deklarieren und dann, abwechselnd, zehn Aufrufe von **scanf** und **printf** programmieren.
- Man kann zehn Variablen deklarieren, die man zunächst liest und dann schreibt.

Dass beides keine realistische Lösung ist, sieht man, wenn man die Aufgabe etwas verallgemeinert: Ein Programm soll zehn (**int**-)Zahlen lesen und wieder schreiben. Es soll sich ohne größeren Aufwand auf beliebige andere Anzahlen umstellen lassen.

Die Lösung bringt eine *Schleife*: Man programmiert die Ein- und Ausgabe *einer* Zahl

und bettet diese Anweisungen in ein Sprachkonstrukt ein, das dafür sorgt, dass sie mehrfach ausgeführt werden. Höhere Programmiersprachen kennen in der Regel verschiedene Schleifen-Anweisungen, die sich, von Formalismen abgesehen, darin unterscheiden, *wie* die Anzahl der erforderlichen Wiederholungen bestimmt wird.

C kennt drei Schleifenanweisungen. Als erstes Beispiel soll hier die `while`-Anweisung dienen.

```
while (bedingung) {
    anweisungsfolge
}
```

Vor jedem Schleifendurchlauf wird die Bedingung *bedingung* ausgewertet. Ist das Ergebnis *wahr* (d.h. ungleich 0), so wird die Anweisungsfolge der Schleife ausgeführt. Ist das Ergebnis dagegen *falsch* (d.h. gleich 0), so wird zu der Anweisung übergegangen, die dem *Schleifenrumpf* folgt, d.h. zu der ersten Anweisung, die nicht mehr zur Anweisungsfolge der Schleife gehört.

Damit können wir die gestellte Aufgabe realisieren. Aufgrund der modifizierten Aufgabenstellung sollte auch schon klar sein, dass die Anzahl der zu kopierenden Zahlen als benannte Konstante festgelegt werden sollte. Die Lösung kann dann so aussehen:

```
#include <stdio.h>

#define ANZAHL 10                /* Anzahl Wiederholungen */

/*= Hauptprogramm =====*/
int main(void) {
    int i = ANZAHL, wert;

    printf("Geben Sie die Zahlen ein:\n");

    while (i > 0) {               /* Schleife */
        scanf("%d", &wert);      /* eine Zahl lesen */
        printf("%d", wert);      /* gelesene Zahl schreiben */
        i = i - 1;               /* Anzahl verringern */
    }
    return 0;
}
```

Quelltext 3.2: Wiederholt Zahlen einlesen und ausgeben mit einer Schleife

Diese Lösung erfüllt offensichtlich auch die erweiterte Aufgabenstellung: Sollen 9, 11 oder irgendeine andere Anzahl von Zahlen gelesen und geschrieben werden, so muss nur der Wert in der `#define`-Direktive entsprechend abgeändert und dann das Programm neu übersetzt und gebunden werden.

3.5 Wertzuweisung

Auf eine neue Anweisung, die eben bereits stillschweigend verwendet wurde, muss noch kurz eingegangen werden: Die *Wertzuweisung*

```
i = i - 1;
```

Sie bewirkt, dass der Ausdruck rechts vom Gleichheitszeichen berechnet und der resultierende Wert in der Variablen links vom Gleichheitszeichen gespeichert wird. Diese Beschreibung macht bereits deutlich, dass man eine Wertzuweisung nicht mit einer Gleichung im Sinne der Mathematik verwechseln darf.

Der Ausdruck auf der rechten Seite kann wie eine mathematische Formel aus mehreren Operanden und Operatoren bestehen. Als Operatoren stehen uns (zunächst) die „üblichen“ mathematischen Operatoren + (Addition), - (Subtraktion), * (Multiplikation) und / (Division) zur Verfügung. Hinzu kommt der Operator % (Rest bei ganzzahliger Division), der bereits in der Einführung angesprochen wurde.

Im Detail werden wir uns im nächsten Abschnitt mit den Wertzuweisungen befassen.

3.6 Zeilenorientierte Verarbeitung

Probiert man das letzte Programm einmal aus, so stellt man fest, dass man damit ein mittelpträchtiges Durcheinander auf dem Bildschirm erzeugen kann, wenn man mehrere Zahlen durch Leerzeichen getrennt auf einer Zeile eingibt. Letztlich tut es aber, was es soll. Woran liegt das vermeintliche Chaos?

Der Bildschirm hat, wenn man es genau betrachtet, eine Doppelfunktion: Einerseits werden auf ihm die Tastatur-Anschläge protokolliert; andererseits dient er als Standardausgabe für die Programme. Der Hintergrund für ein eventuelles Chaos ist die *gepufferte Eingabe*: Die Tasten, die man auf der Tastatur anschlägt, werden zwar sofort *geecho*t, d.h. auf dem Bildschirm protokolliert – einem Programm werden sie allerdings erst zur Verfügung gestellt, wenn die ENTER-Taste gedrückt wird (was als Echo einen Zeilenvorschub ergibt). Unser Beispielprogramm wartet also zunächst auf das Drücken der ENTER-Taste. Enthält die Zeile, die es bekommt, mehrere Zahlen, so liest und schreibt es diese, ohne neuerliche Eingabe zu erwarten, auch wieder in eine Zeile. (Da das Ausgabeformat keinerlei Trennung zwischen den Werten vorsieht, erscheinen die Werte bei der Ausgabe auch unmittelbar hintereinander!) Ist die Maximalzahl der zu kopierenden Zahlen noch nicht erreicht, bleibt die Ausgabe direkt hinter dem letzten geschriebenen Wert stehen – jetzt wird wieder Eingabe erwartet.

Übersichtlicher wird es, wenn man jeden Wert in eine eigene Zeile ausgibt und den Benutzer bittet, jeden Eingabewert mit ENTER abzuschließen. Außerdem kann man dem Benutzer noch Hinweise zur korrekten Eingabe geben.

```
#include <stdio.h>

#define ANZAHL 10                /* Anzahl Wiederholungen */

/*= Hauptprogramm =====*/
int main (void) {
    int i = ANZAHL, wert;

    printf("Geben Sie die Zahlen ein. ");
    printf("Bitte jeweils mit ENTER abschliessen!\n");

    while (i > 0) {                /* Schleife */
        printf("Noch %d Zahlen: ", i);
        scanf("%d", &wert);        /* eine Zahl lesen */
        printf("%d\n", wert);      /* gelesene Zahl schreiben */
        i = i - 1;                 /* Anzahl verringern */
    }
    return 0;
}
```

Quelltext 3.3: Zahlen kopieren mit Vorschüben

Eine Möglichkeit, den Benutzer dazu zu zwingen, pro Zeile nur einen Wert einzugeben, haben wir derzeit noch nicht. (Dieses „Zwingen“ könnte ohnehin auch nur darin bestehen, dass wir aus jeder Eingabezeile nur den ersten Wert interpretieren und eventuelle weitere Werte ignorieren.)

3.7 Lesen bis zum Ende

Die Aufgabenstellung wird noch einmal modifiziert: Das Programm soll so lange `int`-Zahlen lesen und wieder schreiben, wie der Benutzer das wünscht. Oder, in der Terminologie der Datenverarbeitung: Das Programm soll so lange `int`-Zahlen lesen und wieder schreiben, bis das Ende der Eingabedatei erreicht ist. Dazu müssen wir zunächst zwei Fragen klären:

1. Wie kann der Benutzer anzeigen, dass die Datei zu Ende ist?
2. Wie erfährt das Programm diese Information?

Die Antwort auf die erste Frage ist einfach: Tippt der Benutzer unter UNIX `C-d`, so wird das vom Betriebssystem als Dateiende interpretiert.

Die Antwort auf die zweite Frage ist auch nicht viel komplizierter, nur muss zur Erklärung etwas weiter ausgeholt werden. Wir erinnern uns daran, dass C nur Funktionen kennt. Bisher ist der Funktionswert der verwendeten Funktionen `scanf` und `printf` jedoch noch nicht in Erscheinung getreten. Er liefert jedoch die Antwort auf die Frage: `scanf` liefert als Funktionswert die Anzahl der gelesenen und erfolgreich umgewandelten Werte; tritt jedoch bereits *vor* dem Lesen und der Umwandlung des ersten Wertes ein Fehler ein (und das Dateiende zählt in diesem Sinne als Fehler), so ist der Funktionswert `EOF` (*End Of File*).

Offensichtlich muss `EOF` ein negativer, ganzzahliger Wert sein, da alle anderen möglichen Funktionswerte ganzzahlig und nichtnegativ sind. Häufig ist der Wert `-1`, aber das wird vom Standard nicht vorgeschrieben. Die benannte Konstante `EOF` ist im übrigen wie `scanf` und `printf` in `stdio.h` deklariert.

Wie kann man den Funktionswert nutzen? Zur Verfügung steht, neben anderen, der *Vergleichsoperator* `!=` (*ungleich*), mit dem wir den Funktionswert mit der Konstanten `EOF` vergleichen können. Das Resultat ist *wahr* bzw. *falsch*, je nachdem, ob die Relation des Operators zwischen den beiden Operanden besteht oder nicht.

Damit können wir die neue Programmvariante realisieren:

```
#include <stdio.h>

/*= Hauptprogramm =====*/
int main(void) {
    int wert, anzahl = 0;

    printf("Geben Sie die Zahlen ein:\n");

    while (scanf("%d", &wert) != EOF) { /* Zahl lesen */
        anzahl = anzahl + 1;
        printf("Der %d. Wert: %d\n", anzahl, wert);
    }
    return 0;
}
```

Quelltext 3.4: Eingabe bis zum Ende der Eingabe (`EOF`) verarbeiten

Zwei Anmerkungen noch:

- Der Funktionswert von `scanf` erlaubt nicht nur die Erkennung des Dateieindes, sondern auch die Erkennung anderer Fehler bei der Eingabe: Ist der Funktionswert kleiner als die Anzahl der Prozentzeichen im Format und der Variablen in der Argumentliste, so muss etwas schief gelaufen sein. Sollen etwa 10 Werte gelesen werden und ist der Funktionswert 8, so kann man daraus rückschließen, dass beim Lesen des neunten Wertes etwas schief gegangen ist.

EOF wird nur dann geliefert, wenn bereits *vor* dem Lesen des ersten Wertes etwas schief geht. Tritt *beim* Lesen des ersten Wertes ein Fehler ein, so ist der Funktionswert 0. Statt mit EOF sollte man den Funktionswert von `scanf` in unserem Beispiel besser mit 1 vergleichen (`scanf (...) == 1`).

- Auch `printf` liefert einen wohldefinierten Funktionswert. Der Standard schreibt vor:
 - Bei korrektem Abschluss der Ausgabe ist der Funktionswert die Anzahl der geschriebenen Zeichen.
 - Bei fehlerhaftem Abschluss ist der Funktionswert negativ. Den Implementatoren ist es freigestellt, unterschiedliche Fehler durch unterschiedliche Werte zu kennzeichnen.

3.8 Alternativen

Neben der Bildung von Schleifen gibt es eine zweite grundlegende Programmiertechnik, nämlich die *Auswahl aus Alternativen*. Realisiert wird sie durch *Verzweigungsanweisungen*.

Wird bei einer Schleife eine Folge von Anweisungen mehrfach (oder ggf. auch keinmal) ausgeführt, so kann man mit den Verzweigungsanweisungen erreichen, dass in Abhängigkeit von bestimmten Bedingungen aus mehreren Anweisungsfolgen eine ausgewählt und ausgeführt wird.

Die wichtigste Verzweigungsanweisung in C ist die `if`-Anweisung:

```
if (bedingung) {
    anweisungsfolge1
}
[ else {
    anweisungsfolge2
} ]
```

Je nachdem, ob die Auswertung von *bedingung* den Wert *wahr* oder *falsch* ergibt, wird entweder *anweisungsfolge1* oder *anweisungsfolge2* ausgeführt.

Die Vergleichsoperatoren `!=` und `==` haben wir eben bereits kennengelernt. Daneben gibt es die vier Vergleichsoperatoren `<`, `<=`, `>` und `>=`, die in ihrer Bedeutung selbsterklärend sind. Bei `==` muss man allerdings sehr aufpassen! Schreibt man nur ein Gleichheitszeichen, so akzeptiert der Compiler das auch – nur ist die Bedeutung eine *vollständig* andere!

Als Beispiel betrachten wir folgende Aufgabe: Der Benutzer soll eine positive ganze Zahl eingeben; nicht positive Zahlen sollen zurückgewiesen werden. In der Praxis kommt diese Aufgabe sicher nur als Bestandteil umfangreicherer Aufgaben vor; wir wollen sie hier aber für sich lösen.

```
#include <stdio.h>

/*= Hauptprogramm =====*/
int main(void) {
```

```

int wert = 0;

printf("Das Programm wird durch die Eingabe einer"
      "positiven Zahl beendet.\n");

while (wert <= 0) {
    printf("Geben Sie eine Zahl ein: ");
    if (scanf("%d", &wert) != 1) {          /* Zahl lesen          */
        printf("Eingabefehler!\n");        /* Fehlerbehandlung */
        return 1;
    }
    else {
        if (wert <= 0) {                    /* Eingabe ok ?      */
            printf("Falsch - noch einmal!\n"); /* nein ! */
        }
        else {
            printf("Einverstanden!\n");      /* ja !      */
        }
    }
}
return 0;
}

```

Quelltext 3.5: Auswahl aus Alternativen mit `if` und `else`

Hier sehen wir gleich noch etwas: Schleifen und Verzweigungsanweisungen dürfen *geschachtelt* werden.

3.9 Felder

In mathematischen Formeln werden oft Vektoren oder Matrizen verwendet; in anderen Problemkreisen hat man häufig Tabellen. Programmiersprachen stellen entsprechende Datenstrukturen unter dem Oberbegriff *Felder* (oder auch *array*) zur Verfügung. Der sprachlichen Einfachheit halber werden eindimensionale Felder in der Regel als *Vektoren* und zweidimensionale Felder als *Matrizen* bezeichnen.

Das Wesen eines Feldes ist, dass es aus einer festen Anzahl von *Elementen* (*Komponenten*) besteht, die sämtlich denselben Typ besitzen.

In C deklariert man Vektoren, indem man dem Namen des Feldes in der Variablendeklaration die gewünschte Komponentenzahl nachstellt, eingeschlossen in eckige Klammern. Die Deklaration

```
double v[5];
```

oder besser

```
#define LAENGE 5
...
double v[LAENGE];
```

besagt so: *v* ist ein Vektor (eindimensionales Feld) mit 5 Komponenten. Die Dimensionierung *muss* mit Konstanten erfolgen – allerdings sind die benannten Konstanten eben auch Konstanten.

Ansprechen kann man einzelne Komponenten eines Vektors, indem man seinem Namen den *Index* der entsprechenden Komponente nachstellt, erneut eingeschlossen in eckige Klammern. Dabei muss man beachten: Die Numerierung der Komponenten beginnt *immer*

bei 0, so dass entsprechend der höchste (erlaubte) Index um 1 kleiner ist als die Länge des Vektors. In unseren Beispiel hat `v` also die Komponenten `v[0]`, `v[1]`, ..., `v[4]`.

Die Indizes der Komponenten müssen keine Konstanten sein. Erlaubt sind (u.a.) auch Variablen. Klar sollte sein, dass diese Variablen einen ganzzahligen Typ besitzen müssen und dass ihre Werte innerhalb des Indexbereiches des Vektors liegen müssen.

Als Beispiel betrachten wir ein Programm, das zwei `double`-Vektoren liest, ihr Skalarprodukt berechnet und das Resultat ausgibt:

```
#include <stdio.h>

#define LAENGE 5

/*= Hauptprogramm =====*/
int main(void) {
    double v1[LAENGE], v2[LAENGE], prod; /* Variablendekl. */
    int i;

    printf("1. Vektor eingeben!\n");      /* 1. Vektor lesen */
    i = 0;
    while (i < LAENGE) {
        scanf ("%lf", &v1[i]);
        i = i + 1;
    }

    printf("2. Vektor eingeben!\n");      /* 2. Vektor lesen */
    i = 0;
    while (i < LAENGE) {
        scanf ("%lf", &v2[i]);
        i = i + 1;
    }

    i = 0;                                /* Skalarprodukt */
    prod = 0;                             /* berechnen */
    while (i < LAENGE) {                  /* und ausgeben */
        prod = prod + v1[i] * v2[i];
        i = i + 1;
    }
    printf("Das Skalarprodukt ist: %f\n", prod);

    return 0;
}
```

Quelltext 3.6: Arbeit mit Vektoren: Berechnen eines Skalarproduktes

Das sollte zunächst ausreichen, um Ihnen die Verwendung von (eindimensionalen) Feldern zu erlauben.

